Raushan Kumar

100 DATA ENGINEERING SQL INTERVIEW QUESTIONS

RAUSHAN KUMAR

Raushan Kumar
https://www.linkedin.com/in/raushan-kumar-553154297/

100 DATA ENGINERING INTERVIEW SQL PROGRAMS

When preparing for a data engineering interview, it's crucial to focus on SQL skills, as SQL is the backbone for most data operations.

Below are 100 of the most important SQL programs or queries that you should be familiar with, broken down into categories like data retrieval, aggregation, joins, subqueries, window functions, and more.

1. Basic Data Retrieval

1. Select all records from a table

```
SELECT * FROM employees;
```

2. Select specific columns from a table

```
SELECT first_name, last_name FROM employees;
```

3. Filter records using WHERE

```
SELECT * FROM employees WHERE department = 'HR';
```

4. Use AND/OR to combine conditions

```
SELECT * FROM employees WHERE department = 'HR' AND age > 30;
```

5. Use IN to match values from a list

```
SELECT * FROM employees WHERE department IN ('HR',
'Engineering');
```

6. Use BETWEEN to filter range of values

```
SELECT * FROM employees WHERE salary BETWEEN 50000 AND
100000;
```

7. Use LIKE for pattern matching

```
SELECT * FROM employees WHERE first_name LIKE 'A%';
```

8. Select distinct values from a column

```
SELECT DISTINCT department FROM employees;
```

9. Limit the number of records returned

```
SELECT * FROM employees LIMIT 10;
```

10. Use ORDER BY for sorting

```
SELECT * FROM employees ORDER BY salary DESC;
```

---

2. Aggregation

11. Count the number of records

```
SELECT COUNT(*) FROM employees;
```

12. Find the maximum value in a column

```
SELECT MAX(salary) FROM employees;
```

13. Find the minimum value in a column

```
SELECT MIN(salary) FROM employees;
```

14. Calculate the average of a column

```
SELECT AVG(salary) FROM employees;
```

15. Sum values in a column

```
SELECT SUM(salary) FROM employees;
```

16. Group data by a column

```
SELECT department, COUNT(*) FROM employees GROUP BY
department;
```

17. Filter grouped data using HAVING

```
SELECT department, AVG(salary) FROM employees GROUP BY
department HAVING AVG(salary) > 60000;
```

BALAJI KUMAR

18. Find the total salary by department

```
SELECT department, SUM(salary) FROM employees GROUP BY department;
```

19. Find the highest-paid employee in each department

```
SELECT department, MAX(salary) FROM employees GROUP BY department;
```

20. Find the average salary per department and sort

```
SELECT department, AVG(salary) FROM employees GROUP BY department ORDER BY AVG(salary) DESC;
```

---

3. Joins

21. Inner join two tables

```
SELECT e.first_name, d.department_name
FROM employees e
JOIN
departments d
ON e.department_id = d.department_id;
```

22. Left join two tables

```
SELECT e.first_name, d.department_name
FROM employees e
LEFT JOIN
departments d
ON e.department_id = d.department_id;
```

23. Right join two tables

```
SELECT e.first_name, d.department_name
FROM employees e
RIGHT JOIN
departments d
ON e.department_id = d.department_id;
```

24. Full outer join two tables

```
SELECT e.first_name, d.department_name
FROM employees e
FULL OUTER JOIN
departments d
ON e.department_id = d.department_id;
```

25. Self join

```
SELECT e1.first_name AS Employee, e2.first_name AS Manager
FROM employees e1
JOIN
employees e2
ON e1.manager_id = e2.employee_id;
```

---

4. Subqueries

26. Use a subquery in WHERE

```
SELECT * FROM employees WHERE department_id = (SELECT
department_id FROM departments WHERE department_name =
'HR');
```

27. Subquery with IN

```
SELECT * FROM employees WHERE department_id IN (SELECT department_id FROM departments WHERE location = 'NY');
```

28. Subquery with EXISTS

```
SELECT * FROM employees e

WHERE EXISTS (

SELECT 1 FROM departments d WHERE e.department_id = d.department_id AND d.location = 'NY');
```

29. Correlated subquery

```
SELECT e.first_name, e.salary

FROM employees e

WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id);
```

30. Subquery with NOT IN

```
SELECT * FROM employees WHERE department_id NOT IN (SELECT department_id FROM departments WHERE location = 'NY');
```

---

5. Window Functions

31. Rank employees by salary

```
SELECT first_name, salary, RANK() OVER (ORDER BY salary DESC) AS salary_rank FROM employees;
```

32. Calculate running total of salary

```
SELECT first_name, salary, SUM(salary) OVER (ORDER BY salary) AS running_total FROM employees;
```

33. Calculate moving average of salary

```
SELECT first_name, salary, AVG(salary) OVER (ORDER BY salary
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
moving_avg FROM employees;
```

34. Find the highest salary in each department using PARTITION BY

```
SELECT department, first_name, salary, RANK() OVER (PARTITION BY
department ORDER BY salary DESC) AS dept_rank FROM employees;
```

35. Find cumulative salary by department

```
SELECT department, first_name, salary, SUM(salary) OVER
(PARTITION BY department ORDER BY salary) AS cumulative_salary
FROM employees;
```

---

6. Case Statements and Conditional Logic

36. Use CASE for conditional logic

```
SELECT first_name, salary,
CASE
WHEN salary > 70000 THEN 'High'
WHEN salary BETWEEN 50000 AND 70000 THEN 'Medium'
ELSE 'Low'
END AS salary_category
FROM employees;
```

37. Using CASE with GROUP BY

```
SELECT department, COUNT(CASE WHEN salary > 70000 THEN 1
END) AS high_salary_count FROM employees GROUP BY
department;
```

38. Nested CASE statement

```
SELECT first_name, salary,
CASE
WHEN salary > 70000 THEN
        CASE WHEN department = 'HR' THEN 'HR High Salary'
        ELSE 'Other High Salary'
        END
ELSE 'Low Salary'
END AS salary_type
FROM employees;
```

---

7. String Functions

39. Concatenate two columns

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;
```

40. Substring a column value

```
SELECT SUBSTRING(first_name, 1, 3) AS short_name FROM employees;
```

41. Convert string to uppercase

```
SELECT UPPER(first_name) FROM employees;
```

42. Convert string to lowercase

```
SELECT LOWER(last_name) FROM employees;
```

43. Length of a string

```
SELECT LENGTH(first_name) FROM employees;
```

44. Replace substring within a string

```
SELECT REPLACE(first_name, 'John', 'Mike') FROM employees;
```

45. Find the position of a substring

```
SELECT POSITION('HR' IN department) FROM employees;
```

---

8. Date and Time Functions

46. Current date

```
SELECT CURDATE();
```

47. Current time

```
SELECT CURTIME();
```

48. Add days to a date

```
SELECT DATE_ADD(CURDATE(), INTERVAL 10 DAY);
```

49. Subtract days from a date

```
SELECT DATE_SUB(CURDATE(), INTERVAL 5 DAY);
```

50. Extract year from a date

```
SELECT YEAR(hire_date) FROM employees;
```

51. Extract month from a date

```
SELECT MONTH(hire_date) FROM employees;
```

52. Extract day from a date

```
SELECT DAY(hire_date) FROM employees;
```

53. Format date

```
SELECT DATE_FORMAT(hire_date, '%Y-%m-%d') FROM employees;
```

54. Calculate age based on date of birth

```
SELECT first_name, DATEDIFF(CURDATE(), birth_date) / 365 AS age
FROM employees;
```

RAJESH KUMAR

55. Find the difference in days between two dates

```
SELECT DATEDIFF(CURDATE(), hire_date) AS days_since_hired FROM employees;
```

---

9. Data Modification

56. Insert data into a table

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('John', 'Doe', 'Engineering', 70000);
```

57. Update data in a table

```
UPDATE employees SET salary = 75000 WHERE first_name = 'John';
```

58. Delete records from a table

```
DELETE FROM employees WHERE first_name = 'John' AND last_name = 'Doe';
```

59. Insert multiple rows into a table

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES
('Alice', 'Smith', 'HR', 55000),
('Bob', 'Johnson', 'Engineering', 60000);
```

60. Replace data in a table (similar to INSERT or UPDATE)

```
REPLACE INTO employees (employee_id, first_name, last_name, department, salary)
VALUES (1, 'John', 'Doe', 'Engineering', 70000);
```

---

10. Temporary Tables and Advanced Concepts

61. Create and use a temporary table

```
CREATE TEMPORARY TABLE temp_employees AS SELECT * FROM employees;
```

62. Use IF statement in a query

```
SELECT first_name, IF(salary > 70000, 'High', 'Low') AS salary_level FROM employees;
```

63. Use CASE with JOIN

```
SELECT e.first_name, e.last_name, e.salary,  d.department_name,

CASE

        WHEN e.salary > 80000 THEN 'High Salary'

        WHEN e.salary BETWEEN 50000 AND 80000 THEN 'Medium Salary'

        ELSE 'Low Salary'

 END AS salary_status

FROM employees e

JOIN departments d ON e.department_id = d.department_id;
```

64. Use WITH clause (Common Table Expressions, CTE)

```
WITH dept_avg_salary AS (

SELECT department_id, AVG(salary) AS avg_salary FROM employees

GROUP BY department_id)

SELECT e.first_name, e.salary, d.avg_salary FROM employees e JOIN dept_avg_salary d ON e.department_id = d.department_id;
```

65. Handling NULL values using COALESCE

```
SELECT first_name, COALESCE(salary, 0) FROM employees;
```

---

11. Database Design and Constraints

66. Create a table with constraints

```
CREATE TABLE employees (
employee_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50),
last_name VARCHAR(50),
salary DECIMAL(10,2) CHECK (salary >= 0)
);
```

67. Alter a table to add a new column

```
ALTER TABLE employees ADD COLUMN hire_date DATE;
```

68. Add a foreign key constraint

```
ALTER TABLE employees ADD CONSTRAINT fk_department
FOREIGN KEY (department_id)
REFERENCES departments(department_id);
```

69. Drop a column from a table

```
ALTER TABLE employees DROP COLUMN hire_date;
```

70. Rename a table

```
RENAME TABLE employees TO staff;
```

---

12. Indexes and Optimization

71. Create an index on a column

```
CREATE INDEX idx_salary ON employees(salary);
```

72. Drop an index

```
DROP INDEX idx_salary ON employees;
```

73. Show all indexes for a table

```
SHOW INDEX FROM employees;
```

74. Explain query execution plan

```
EXPLAIN SELECT * FROM employees WHERE salary > 60000;
```

75. Use LIMIT with OFFSET

```
SELECT * FROM employees ORDER BY hire_date DESC LIMIT 10
OFFSET 20;
```

---

13. Advanced Data Operations

76. Create a view

```
CREATE VIEW high_salary_employees AS
SELECT first_name, last_name, salary
FROM employees
WHERE salary > 70000;
```

77. Drop a view

```
DROP VIEW high_salary_employees;
```

78. Create a stored procedure

```
DELIMITER $$
CREATE PROCEDURE
get_employee_by_id(IN emp_id INT)
BEGIN
SELECT * FROM employees WHERE employee_id = emp_id;
END$$
DELIMITER ;
```

79. Call a stored procedure

```
CALL get_employee_by_id(1);
```

80. Create a trigger

```
DELIMITER $$
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
SET NEW.created_at = NOW();
END$$
DELIMITER ;
```

---

14. Advanced Query Techniques

81. Find duplicate records:

```
SELECT first_name, last_name, COUNT(*)
FROM employees
GROUP BY first_name, last_name HAVING COUNT(*) > 1;
```

82. Find the difference between two tables (using EXCEPT or NOT EXISTS)

```
SELECT * FROM employees

EXCEPT

SELECT * FROM contractors;
```

83. Join multiple tables (e.g., 3 tables)

```
SELECT e.first_name, d.department_name, l.location

FROM employees e

JOIN departments d

ON e.department_id = d.department_id

JOIN locations l

ON d.location_id = l.location_id;
```

84. Find the second-highest salary:

```
SELECT MAX(salary)

FROM employees

WHERE salary < (SELECT MAX(salary) FROM employees);
```

85. List all managers and their employees:

```
SELECT manager_id, GROUP_CONCAT(employee_id)

FROM employees

GROUP BY manager_id;
```

---

15. Optimization and Tuning

86. Use of EXPLAIN for query optimization:

```
EXPLAIN SELECT * FROM employees WHERE department = 'HR';
```

87. Optimize joins using indexes:

```
CREATE INDEX idx_employee_id
ON employees(employee_id);
```

88. Optimize subqueries with joins:

```
SELECT e.first_name, e.salary
FROM employees e
JOIN (SELECT department_id, MAX(salary) AS max_salary FROM
employees GROUP BY department_id) m
ON e.salary = m.max_salary;
```

89. Avoiding SELECT * in large tables:

```
SELECT first_name, last_name FROM employees;
```

90. Optimize date queries with appropriate indexing:

```
CREATE INDEX idx_hire_date ON employees(hire_date);
```

---

16. Data Engineering-Specific Queries

91. Data pipeline SQL for ETL:

```
INSERT INTO final_table (SELECT * FROM raw_data WHERE
processed = 0);
```

92. Identify bottlenecks in data processing:

```
SELECT stage, COUNT(*) FROM pipeline_data WHERE status =
'ERROR' GROUP BY stage;
```

93. Incremental data loading (e.g., using timestamp):

```
SELECT * FROM raw_data WHERE timestamp > '2025-01-01';
```

94. Data partitioning for big data:

```
CREATE TABLE large_table
PARTITION BY RANGE (year(timestamp)) (
PARTITION p2020 VALUES LESS THAN (2021),
PARTITION p2021 VALUES LESS THAN (2022)
);
```

95. Handling schema changes in data:

```
ALTER TABLE data ADD COLUMN new_column INT;
```

---

17. Data Warehousing Concepts

96. Star schema query:

```
SELECT f.sales, d.product_name, d.category
FROM fact_sales f
JOIN dim_product d
ON f.product_id = d.product_id;
```

97. Snowflake schema query:

```
SELECT f.sales, d.product_name, c.category_name
FROM fact_sales f
JOIN dim_product d
ON f.product_id = d.product_id
JOIN dim_category c
ON d.category_id = c.category_id;
```

98. Handling slowly changing dimensions:

```
UPDATE dim_customer SET address = 'New Address' WHERE
customer_id = 1;
```

99. Data aggregation for reporting:

```
SELECT product_name, SUM(sales) AS total_sales

FROM fact_sales

GROUP BY product_name;
```

100. Data transformation in staging:

```
INSERT INTO staging_table (column1, column2) SELECT column1,
CONCAT('Processed_', column2) FROM raw_data;
```

RAUSHAN KUMAR

By: Raushan Kumar

Please follow for more such content:

https://www.linkedin.com/in/raushan-kumar-553154297/